

SQL Master Class – AI-Powered SQL Learner Platform

CS 5200 Database Management Systems – Final Project Report

Group 7

Team Members: Qiannan Shi, Tian Yuan, Xijia Zeng

Date: 2025-12-07

Executive Summary

SQL Master Class is a web-based, role-based SQL learning platform that supports Students, Instructors, and Admins. Students practice problems on real datasets using an in-browser SQL editor and receive instant autograder feedback. An AI SQL Tutor provides guided hints and natural-language-to-SQL assistance. The system stores all submissions and generates personalized analytics, time-on-task summaries, and leaderboards. Instructors view class-level dashboards including pass rates, attempts, average scores, and time-on-task rankings. Admins manage user roles and oversee the problem bank, with planned future support for content/test-case moderation.

Introduction

Learning SQL effectively requires repetition, feedback, and exposure to realistic data. However, learners often struggle to translate natural language intentions into correct SQL, and instructors may lack a unified view of student progress. This project addresses these gaps through a single platform combining practice, autograding, analytics, and LLM-powered assistance.

Project Scope and Goals

Our goals were to:

- Provide a menu-driven web interface for all DB interactions.
- Support distinct workflows for students, instructors, and admins using RBAC.
- Implement autograding with persistent submission history.
- Deliver analytics for both individuals and classes.
- Integrate LLM-driven Open SQL and chat-based assistance.

Phase I – Project Initiation (Revised Summary)

Phase I established the problem statement, high-level architecture, and use cases. We defined a role-based AI-assisted SQL practice platform that bridges plain-English goals and correct SQL. We also drafted a set of realistic analytics and learning queries, and identified the core technology stack (web app + MySQL + LLM integration).

Phase II – Conceptual and Logical Design (Revised Summary)

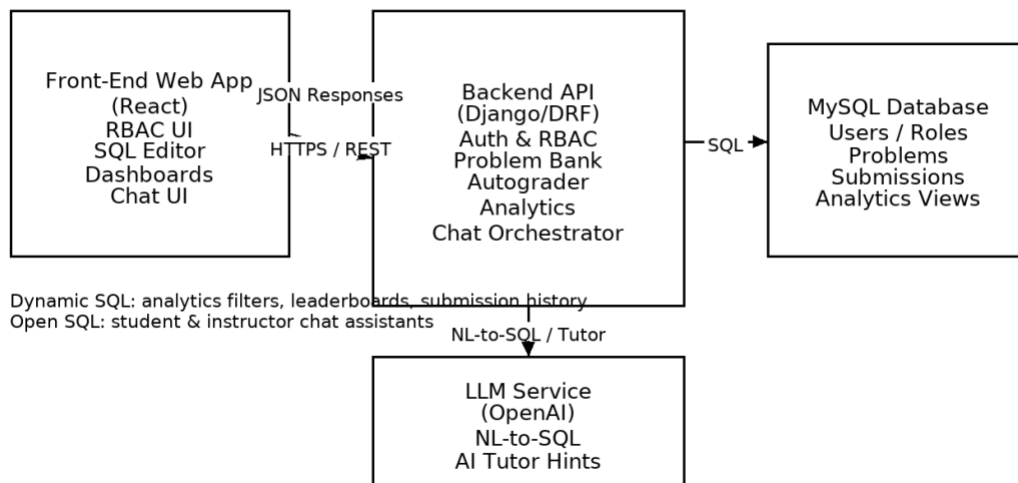
Phase II produced the EER model and translated it into a logical schema. Key entities include USER, ROLE, USER_ROLE, DATASET, DATASET_VERSION, PROBLEM, TAG, PROBLEM_TAG, SUBMISSION, AUTOGRADER_LOG, and leaderboard-related tables. We documented functional dependencies and provided relational algebra for representative analytics queries.

Application Development (Step 14)

Architecture

The application follows a three-tier architecture: a web front-end, a backend API layer, and a MySQL database. The system integrates an LLM service to power the AI SQL Tutor and role-aware chat assistants. Users interact with the DBMS only through the application interface.

SQL Master Class - System Architecture



Dynamic SQL

Dynamic SQL is used extensively in the analytics and practice workflows. Examples include generating queries for filtered submission history, time-on-task calculations per date range, and leaderboard ranking logic. These queries are constructed based on user role, selected filters, and UI-driven parameters.

Open SQL Programming

The Student and Instructor chat assistants support natural-language queries. The LLM converts user intent into SQL, which is then executed against the database with role-based guardrails. The chat experience is also designed to support controlled real-time updates where appropriate.

Student Workflow

- Login → Learner Dashboard
- Browse Problem Bank and open a problem
- Write SQL in editor → Run & Grade
- Use AI SQL Tutor for hints and guidance
- View Analytics: last submissions, time-on-task, leaderboard

- • Ask chat questions (e.g., latest submission)

Instructor Workflow

- • Login → Instructor Dashboard
- • Quick Stats: students, active problems, completion rate
- • Class Analytics: pass rates, attempts, average scores
- • Time-on-task leaderboard
- • Instructor chat questions (e.g., most hard-working student)

Admin Workflow

- • User management: update roles and remove users
- • Problem bank access for future moderation workflows

Team Contributions

We contributed equally in every phase and worked together to finish each task, including requirements, schema design, implementation, data population, application development, and final integration/testing.

Conclusion

SQL Master Class demonstrates an end-to-end database application that integrates role-based learning, autograding, analytics, and LLM-powered Open SQL.

Throughout this project, we deepened our understanding of conceptual-to-physical database design, query design for analytics, and the practical challenges of safely deploying NL-to-SQL features. In details, we gained comprehensive experience in full-stack web development by building an end-to-end learning platform. On the frontend, we worked extensively with React's component-based architecture, implementing state management for user authentication, routing with React Router for role-based navigation, and protected routes to control access based on user roles. The backend development with Django taught me RESTful API design principles, including creating view functions for various endpoints, configuring URL routing. We integrated the React frontend with the Django backend through well-structured REST APIs, handling data flow between client and server while managing authentication states across both layers. The database design aspect proved particularly educational as we worked with a complex MySQL schema containing interconnected tables like User, Problem, Submission, Event, and various analytical views. We learned to navigate the challenges of composite keys and adapted our application logic to work with an existing table structure rather than designing from scratch. We created and utilized database views such as SubmissionHistory and ActiveProblemsWithStats to simplify complex queries and improve performance. My SQL skills improved significantly through implementing analytics queries that employed Common Table Expressions (CTEs), window functions like LAG and OVER for time-gap calculations, and date-based groupings for time-on-task tracking. We also gained practical experience with query optimization

techniques including strategic indexing and using aggregation functions for conditional counting and pivoting data. Schema adaptation taught us the important real-world skill of working within existing database constraints while implementing new features, requiring creative solutions to map column names and structures to application requirements.

For potentially better SQL Master Class platform application, several critical enhancements would improve both security and user experience. Implementing email verification would ensure user authenticity by requiring new registrants to confirm their email addresses through verification links, adding a crucial layer of account security and reducing spam accounts. A discussion forum feature would transform the platform from individual practice into a collaborative learning environment, enabling students to ask questions about specific problems, share insights, receive help from instructors and peers, and build a knowledge base of common SQL challenges and solutions through upvoted answers and threaded discussions. Automated testing would be essential for maintaining code quality and reliability as the platform scales, requiring comprehensive unit tests for backend API endpoints, frontend component tests, integration tests to verify the complete authentication and submission workflows, and end-to-end tests. Performance optimization would address scalability concerns by implementing database query optimization through strategic indexing on frequently queried columns like User_ID and Created_at, ultimately ensuring the platform can handle hundreds of concurrent users without degradation in response times. In conclusion, better output could focus on expanding the problem bank, improving test-case coverage, and adding stronger admin moderation and content quality workflows.